

# **UAB**

## **Universitat Autònoma de Barcelona**

Diseño e implementación de una API de control de acceso para la gestión de usuarios y organitzacions

Primer informe de seguimiento

Igor Ulyanov Melnic  
Tutora: Victoria Montenegro Ruíz

Grado en Ingeniería Informática  
Escuela de Ingeniería  
Curso 2025-2026

## 0. Control del documento

### 0.1 Información del documento

|                     |                              |
|---------------------|------------------------------|
| Identificador       | primerSeguimiento            |
| Autor               | Igor Ulyanov Melnic          |
| Fecha de creación   | 31/3/2026                    |
| Ultima modificacion | 05/4/2026                    |
| Nombre del fichero  | Informe de Seguimiento 1.pdf |

### 0.2 Historial del documento

| Versión | Fecha     | Cambios                                  |
|---------|-----------|------------------------------------------|
| 1.0     | 31/3/2026 | Creación del documento, esbozo inicial   |
| 1.1     | 1/4/2026  | Resumen                                  |
| 1.2     | 2/4/2026  | Seguimiento de la metodologia            |
| 1.3     | 3/4/2026  | Generacion del documento de requisitos   |
| 1.4     | 5/4/2026  | Finalizacion de la primera version final |
| 1.5     | 7/4/2026  | Cronograma                               |
| 1.6     | 11/4/2026 | Anexos                                   |

# Índice

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>0. Control del documento.....</b>             | <b>2</b>  |
| 0.1 Información del documento.....               | 2         |
| 0.2 Historial del documento.....                 | 2         |
| <b>Índice.....</b>                               | <b>3</b>  |
| <b>1. Introducción.....</b>                      | <b>4</b>  |
| 1.1 Resumen.....                                 | 4         |
| 1.2 Seguimiento de la metodología.....           | 4         |
| <b>2. Progreso Cronológico.....</b>              | <b>6</b>  |
| 2.1. Generación del documento de requisitos..... | 6         |
| 2.2. Diseño de la base de datos.....             | 7         |
| 2.3. Implementación de la base de datos.....     | 8         |
| 2.3.1. Docker-compose.....                       | 8         |
| 2.3.2. Scripts de inicialización.....            | 10        |
| <b>3. Bibliografía.....</b>                      | <b>11</b> |

# 1. Introducción

## 1.1 Resumen

El principal objetivo de esta fase ha sido la definición del trabajo a realizar, junto con el diseño preliminar de la base de datos, la preparación del entorno de desarrollo (IDE) y el inicio de la implementación de la aplicación. En primer lugar, se elaboró un documento de requisitos que recoge las funcionalidades principales en forma de historias de usuario, así como una lista detallada de requisitos funcionales y no funcionales.

A partir de esta delimitación del alcance, se diseñó un diagrama de base de datos relacional, el cual ha servido como guía para la posterior implementación de un contenedor Docker basado en una imagen de PostgreSQL. Dicho contenedor se gestiona mediante Docker Compose e incluye una serie de scripts SQL que permiten la creación automática del esquema de la base de datos, así como la inserción de datos iniciales en la primera ejecución del servicio.

El desarrollo del proyecto se ha llevado a cabo siguiendo la metodología Waterfall. El periodo comprendido entre el informe inicial y el primer informe de progreso marca la finalización de la fase de diseño y el inicio de la fase de implementación. Para el seguimiento del trabajo, se ha utilizado un tablero Kanban<sup>[1]</sup> en Trello, cuyo backlog está compuesto principalmente por los requisitos previamente definidos.

En retrospectiva, se ha identificado un problema relevante: la metodología Waterfall no resulta la más adecuada para proyectos que implican el uso de tecnologías o lenguajes de programación desconocidos. Dado el limitado conocimiento práctico en herramientas como Java Spring Boot, Spring Security o Swagger, se ha decidido posponer el diseño formal de la API. Este se abordará una vez se adquiera un mayor dominio de dichas tecnologías, ya que es probable que el diseño evolucione significativamente durante el proceso de aprendizaje.

## 1.2 Seguimiento de la metodología

En el informe inicial se planteó un modelo de desarrollo en cascada complementado con elementos organizativos propios de Kanban. Actualmente, el proyecto se encuentra en la fase de implementación. Las semanas previas al informe inicial correspondieron a la fase de ideación e inicio del proyecto, mientras que la semana posterior a su entrega se dedicó a la fase de diseño. Desde ese momento y hasta la entrega del segundo informe se desarrolla la fase de implementación. Finalmente, se han reservado varias semanas para la verificación y validación del correcto funcionamiento del sistema.

Hasta el momento, la adherencia a esta planificación ha sido completa. Durante la fase de diseño se elaboró el documento de requisitos, junto con el esquema de la base de datos relacional. La fase de implementación se ha iniciado sin retrasos y progresa conforme a lo

previsto. Si bien en la planificación inicial no se definieron con precisión las tareas diarias o entregas parciales, en la práctica se ha adoptado un ritmo de trabajo similar al de metodologías ágiles, asignando semanalmente tareas del backlog para su desarrollo.

No obstante, se han identificado ciertas limitaciones en este enfoque, especialmente durante la fase de diseño. Debido al conocimiento limitado de tecnologías como Swagger, Spring Boot o Spring Security, se ha decidido posponer el diseño formal del código de la API. Este diseño se realizará fuera del orden establecido en la metodología inicial, con el objetivo de permitir una evolución iterativa a medida que se adquiere mayor dominio de dichas herramientas, facilitando así la creación de una solución más práctica y ajustada a la realidad del desarrollo.

Asimismo, se han observado beneficios imprevistos en la metodología adoptada. La ausencia de una planificación estricta a corto plazo ha permitido una mayor flexibilidad en la distribución de la carga de trabajo semanal, adaptándose mejor a las necesidades del desarrollo.

En la planificación inicial se propuso el uso de un tablero Kanban para la gestión del backlog, el cual se ha implementado mediante la herramienta Trello de Atlassian<sup>[2]</sup>, tal y como se muestra en la Figura 1. Este tablero recoge el backlog principalmente en forma de requisitos funcionales, junto con algunas tareas de carácter organizativo que no se consideran funcionalidades del producto final. Además, se han añadido columnas específicas para registrar problemas tecnológicos encontrados y sus posibles soluciones, lo que permite su seguimiento sin interferir con el flujo principal de desarrollo.

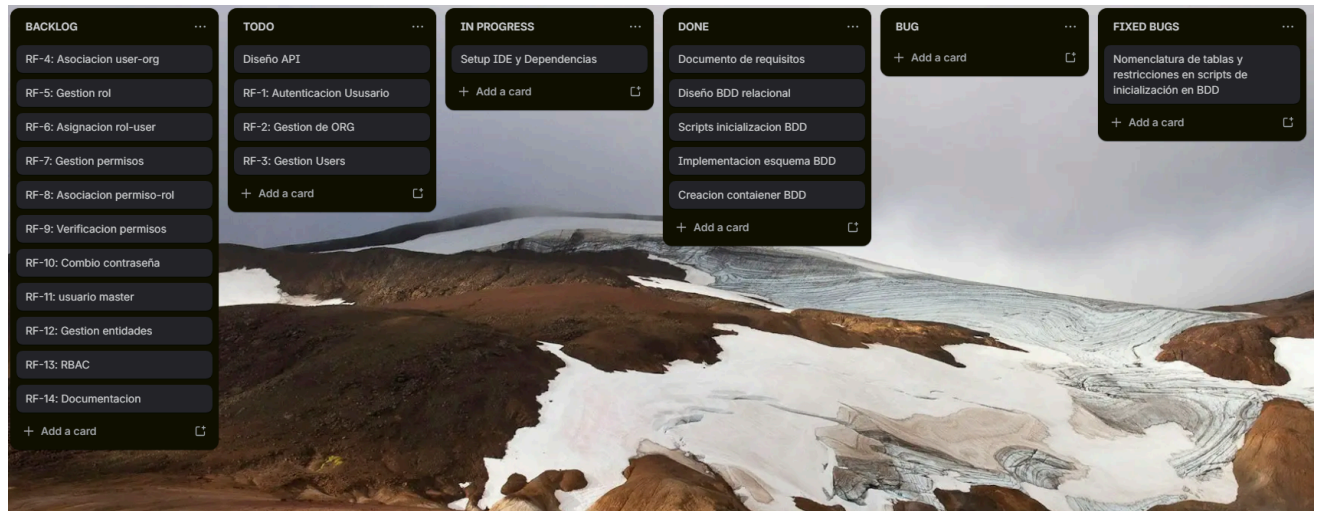


Fig 1. Tablero kanban

## 2. Progreso Cronológico

En la Figura 2 se muestra el progreso del proyecto en relación con el cronograma inicial propuesto. Las fases completadas hasta el hito correspondiente a la entrega del segundo informe de seguimiento aparecen resaltadas en color amarillo. Estas abarcan las etapas de ideación, diseño e implementación de la base de datos dentro del marco de la metodología Waterfall.

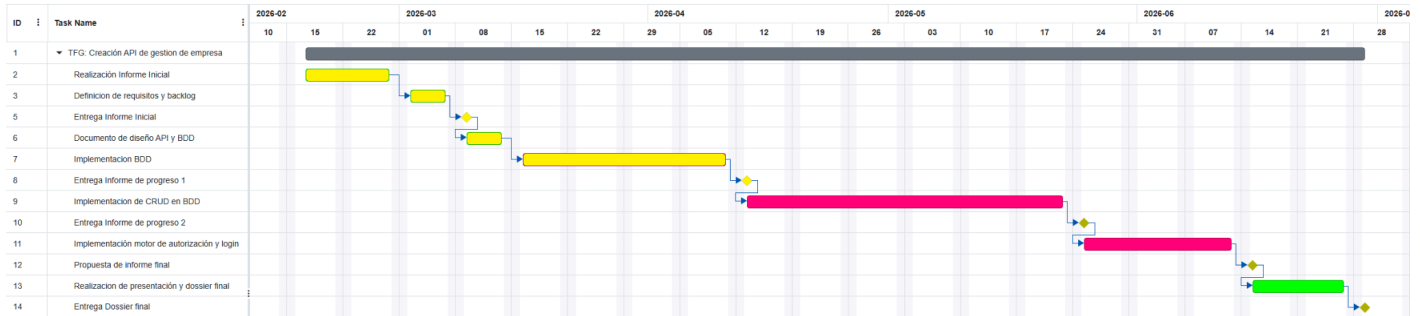


Fig 2: Progreso reflejado en el diagrama de Gantt

A continuación, en este documento se detallan los distintos avances realizados a lo largo de las semanas hasta alcanzar dicho hito.

### 2.1. Generación del documento de requisitos

La finalidad de esta primera etapa del diseño es definir el alcance del proyecto y establecer con claridad qué funcionalidades formarán parte del producto final. Estos requisitos han sido definidos de forma autónoma y constituyen el núcleo mínimo del sistema. El objetivo de este documento no es limitar el desarrollo, sino establecer un producto mínimo viable (MVP) hacia el cual orientar el trabajo. No obstante, no se descarta la incorporación de nuevas funcionalidades no contempladas inicialmente, siempre que se consideren necesarias y coherentes con el sistema.

Todos los requisitos funcionales son imprescindibles para garantizar el correcto funcionamiento del sistema. Sin embargo, algunos de ellos representan los elementos más críticos y, por tanto, deberán ser priorizados durante el desarrollo de la API. Estos son los siguientes:

- **RF1 – Autenticación de usuarios:** El sistema debe permitir a los usuarios iniciar sesión, con el fin de delimitar y controlar sus acciones dentro de la aplicación.
- **RF6 – Asignación de roles a usuarios:** Para aportar la propuesta de valor del sistema, cada usuario debe disponer de un rol con un conjunto de permisos asociados, lo cual resulta esencial para implementar el sistema de autorización.
- **RF9 – Verificación de permisos:** Este constituye el núcleo del sistema. Un usuario debe poder consultar si está autorizado a realizar determinadas acciones en función de sus permisos dentro de la organización.

- **RF14 – Documentación de la API:** Dado que uno de los objetivos principales es la usabilidad, resulta imprescindible disponer de una documentación clara, completa y accesible de los endpoints.

Además de las funcionalidades, el sistema debe cumplir una serie de requisitos no funcionales, orientados a garantizar un estándar de calidad que permita considerar el producto como útil y completo. Entre los más relevantes se encuentran:

- **RNF1 – Seguridad:** Las contraseñas almacenadas en la base de datos deben estar debidamente protegidas, y en ningún caso se debe exponer información sensible a usuarios no autorizados.
- **RNF6 – Portabilidad:** El sistema debe ser ligero y fácil de desplegar. Para ello, se utilizará una arquitectura basada en contenedores.
- **RNF7 – Persistencia de datos:** Se deben configurar volúmenes en el almacenamiento local que garanticen la persistencia de los datos, ya que los contenedores no la proporcionan de forma nativa.

Por último, el documento incluye una serie de historias de usuario. Estas consisten en descripciones breves de situaciones típicas en las que un usuario interactúa con la aplicación. Cada historia se compone de un actor principal, una acción que desea realizar y el resultado esperado. Estas historias sirven como casos de prueba de fácil ejecución que permiten validar el funcionamiento esencial del sistema. Aunque no constituyen una descripción exhaustiva de todas las posibles interacciones, sí establecen una línea base que el desarrollo debe alcanzar. Los listados de requisitos e historias de usuario se encuentran en el [anexo A](#).

## 2.2. Diseño de la base de datos

La base de datos diseñada cumple cuatro funciones principales: almacenar la información necesaria para la autenticación de usuarios, gestionar la estructura organizativa, registrar los roles asociados a cada organización y mantener las relaciones entre estos elementos. En la Figura 3 se muestra el esquema relacional correspondiente a este diseño.

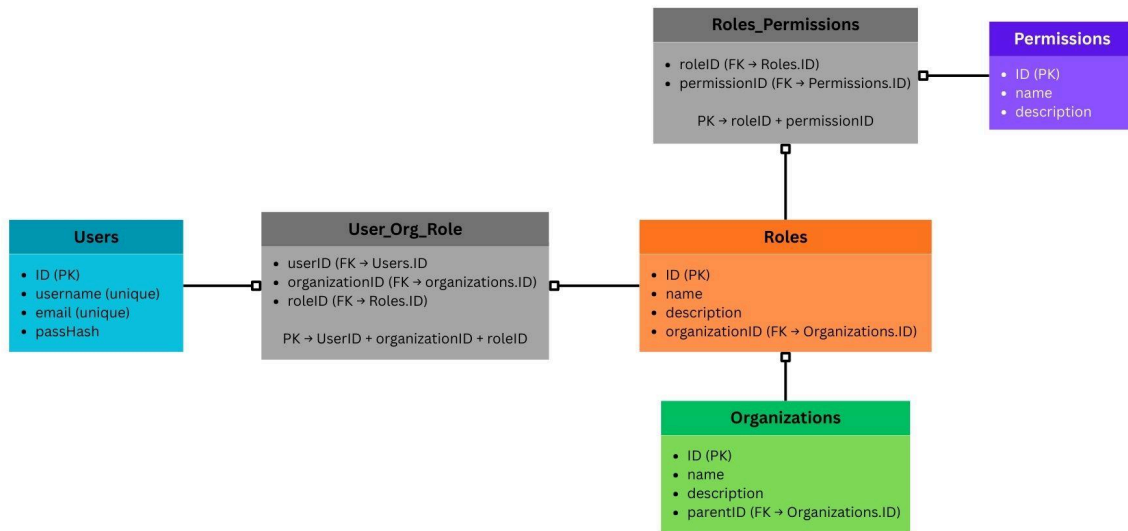


Fig. 3: Esquema de la BDD relacional

Las tablas principales del sistema son **Users**, **Roles**, **Organizations** y **Permissions**, encargadas de almacenar, respectivamente, la información de usuarios, roles, estructura organizativa y permisos. Cada una de estas entidades dispone de un identificador único (ID) como clave primaria, además de los atributos necesarios para describir cada objeto.

La relación entre **Organizations** y **Roles** es de tipo uno a muchos (1:N), lo que implica que cada organización puede definir múltiples roles, pero cada rol pertenece exclusivamente a una única organización. Esta decisión de diseño responde a la naturaleza multi-tenant del sistema, donde cada organización debe gestionar sus propios roles sin compartirlos con otras, garantizando así el aislamiento de datos.

Por otro lado, las relaciones entre usuarios, organizaciones y roles, así como entre roles y permisos, son de tipo muchos a muchos (N:M). Estas relaciones se implementan mediante tablas intermedias (**User\_Org\_Role** y **Roles\_Permissions**), representadas en el esquema en color gris. Dichas tablas contienen únicamente las claves foráneas de las entidades que relacionan, formando una clave primaria compuesta por la combinación de estos campos.

En conclusión, se trata de un esquema sencillo pero suficiente para cubrir las necesidades fundamentales del sistema. Permite almacenar la información necesaria y establecer las relaciones requeridas para implementar correctamente los mecanismos de autenticación y autorización de usuarios.



## 2.3. Implementación de la base de datos

Para el despliegue de la base de datos se ha optado por el uso de contenedores<sup>[3]</sup>, lo que permite ejecutar la aplicación de forma consistente, reproducible y sencilla en cualquier entorno que soporte esta tecnología. Como herramienta principal se ha seleccionado Docker<sup>[4]</sup>, que facilita la gestión y ejecución de contenedores de manera eficiente.

Además, se ha utilizado Docker Compose<sup>[5]</sup>, que permite orquestar la inicialización de múltiples contenedores de forma simultánea mediante la definición de un único archivo de configuración. Siguiendo el tutorial oficial<sup>[6]</sup> para la creación de este tipo de scripts, se ha implementado la solución descrita a continuación.

### 2.3.1. Docker-compose

```
version: '3.8'

services:
  postgres:
    image: postgres:16
    container_name: data-storage
    restart: always
    environment:
      POSTGRES_DB: access_control_db
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./db:/docker-entrypoint-initdb.d

volumes:
  postgres_data:
```

El anterior fragmento corresponde al archivo de configuración de docker-compose.yml, utilizado para definir y gestionar el despliegue del servicio de base de datos PostgreSQL dentro de un contenedor.

En primer lugar, se especifica la versión del formato de Docker Compose (version: '3.8'), que determina las características disponibles en el archivo de configuración.

A continuación, se define el bloque servicios (services) , dentro del cual se declara el uso de postgres. Este servicio utiliza la imagen oficial postgres:16, lo que garantiza una base estable y

mantenida del sistema gestor de bases de datos. Además, se asigna un nombre al contenedor (`container_name: data-storage`) para facilitar su identificación.

La propiedad `restart: always` indica que el contenedor se reiniciará automáticamente en caso de fallo o si el sistema se reinicia, mejorando así la disponibilidad del servicio.

En el apartado `environment` se configuran las variables de entorno necesarias para la inicialización de la base de datos:

- **POSTGRES\_DB**: nombre de la base de datos que se creará automáticamente.
- **POSTGRES\_USER**: usuario administrador de la base de datos.
- **POSTGRES\_PASSWORD**: contraseña asociada a dicho usuario.

El bloque `ports` establece el mapeo de puertos entre la máquina anfitriona y el contenedor. En este caso, el puerto 5432 del host se enlaza con el puerto 5432 del contenedor, permitiendo el acceso a la base de datos desde el exterior.

En cuanto al almacenamiento, se definen dos volúmenes en la sección `volumes` del servicio:

- **postgres\_data:/var/lib/postgresql/data**: volumen persistente que almacena los datos de la base de datos, evitando su pérdida al eliminar o recrear el contenedor.
- **./db:/docker-entrypoint-initdb.d**: volumen que enlaza un directorio local con el directorio de inicialización del contenedor. Esto permite ejecutar automáticamente scripts SQL en el primer arranque, facilitando la creación del esquema y la carga de datos iniciales.

Finalmente, en la sección `global volumes`, se declara el volumen `postgres_data`, que será gestionado por Docker para garantizar la persistencia de los datos.

En conjunto, este archivo permite desplegar de forma automatizada una instancia de PostgreSQL completamente configurada, asegurando tanto la persistencia de los datos como la inicialización automática del sistema en su primer uso.

### 2.3.2. Scripts de inicialización

En el script `docker-compose.yml` se detalla un volumen con scripts de inicialización. Estos scripts SQL desempeñan un papel fundamental en el despliegue automatizado de la base de datos dentro del entorno containerizado. Gracias a su integración con Docker, se ejecutan automáticamente durante la primera inicialización del servicio, permitiendo no solo la creación del esquema de la base de datos, sino también la inserción de datos iniciales necesarios para

su correcto funcionamiento. Esto garantiza que el sistema pueda desplegarse de forma reproducible y consistente en cualquier entorno, sin necesidad de configuración manual adicional.

El primer script, **01-schema.sql**, íntegro en el [anexo B](#), se encarga de definir la estructura de la base de datos. En él se crean todas las tablas necesarias para el sistema, incluyendo *users*, *organizations*, *roles* y *permissions*, así como las tablas intermedias que modelan las relaciones de tipo muchos a muchos (*roles\_permissions* y *user\_org\_role*). Además, se establecen las claves primarias y foráneas, junto con sus correspondientes restricciones, lo que permite garantizar la integridad referencial de los datos. También se incluyen decisiones de diseño relevantes, como el uso de eliminaciones en cascada o la posibilidad de estructuras jerárquicas en organizaciones mediante una autorreferencia. En conjunto, este script define completamente el modelo relacional sobre el que se apoya el sistema.

El segundo script, **02-init-data.sql**, disponible en el [anexo C](#), tiene como objetivo inicializar la base de datos con un conjunto mínimo de datos necesarios para su funcionamiento. En primer lugar, inserta una lista predefinida de permisos que constituyen la base del sistema de autorización. A continuación, crea una organización inicial del sistema, un rol con privilegios completos (*MASTER*) y asigna todos los permisos a dicho rol. Finalmente, se crea un usuario administrador y se le asigna este rol dentro de la organización del sistema. Este proceso permite disponer desde el inicio de un entorno funcional sobre el que operar y realizar pruebas, evitando la necesidad de configuración manual tras el despliegue.

En conjunto, ambos scripts permiten automatizar completamente la inicialización del sistema: el primero define la estructura y el segundo establece un estado inicial coherente. Esta aproximación no solo mejora la eficiencia del despliegue, sino que también facilita la portabilidad y reproducibilidad del entorno, aspectos clave en el desarrollo basado en contenedores.

### 3. Bibliografía

[1] Atlassian. (s.f.). *¿Qué es un tablero de kanban?* Recuperado el 5 de abril de 2026, de <https://www.atlassian.com/es/agile/kanban/boards>

[2] Atlassian. (s.f.). *Trello*. Recuperado el 2 de abril de 2026, de [Trello](#)

[3] Red Hat. (2021, abril 8). *What is containerization?* Recuperado el 5 de abril de 2026, de <https://www.redhat.com/en/topics/cloud-native-apps/what-is-containerization>

[4] Docker, Inc. (s.f.). *Docker overview*. Recuperado el 5 de abril de 2026, de <https://docs.docker.com/get-started/docker-overview/>

[5] Docker, Inc. (s.f.). *Docker Compose overview*. Recuperado el 5 de abril de 2026, de <https://docs.docker.com/guides/docker-compose/>

[6] Docker, Inc. (s.f.). *Get started with Docker Compose*. Recuperado el 5 de abril de 2026, de <https://docs.docker.com/compose/gettingstarted/>

## 4. Anexos

### Anexo A: Listados de requisitos e historias de usuario

#### Requisitos funcionales

##### **RF1 – Autenticación de usuarios**

El sistema deberá permitir que los usuarios se autenticuen mediante credenciales válidas (usuario y contraseña) y, en caso de éxito, emitir un token JWT para su identificación en posteriores peticiones.

##### **RF2 – Gestión de organizaciones**

El sistema deberá permitir crear, consultar, modificar y eliminar organizaciones dentro de la plataforma.

##### **RF3 – Gestión de usuarios**

El sistema deberá permitir la creación, consulta, modificación y eliminación de usuarios.

##### **RF4 – Asociación de usuarios a organizaciones**

El sistema deberá permitir que un usuario pertenezca a una o varias organizaciones.

##### **RF5 – Gestión de roles**

El sistema deberá permitir crear, modificar, consultar y eliminar roles dentro del sistema.

##### **RF6 – Asignación de roles a usuarios**

El sistema deberá permitir asignar roles a usuarios dentro de una organización determinada.

##### **RF7 – Gestión de permisos**

El sistema deberá tener una serie de permisos predeterminados que representen acciones que pueden realizarse dentro del sistema.

##### **RF8 – Asociación de permisos a roles**

El sistema deberá permitir asociar uno o varios permisos a cada rol.

#### **RF9 – Verificación de permisos (Autorización de usuario)**

El sistema deberá proporcionar un endpoint que permita a aplicaciones externas consultar si un usuario tiene permiso para realizar una acción concreta dentro de una organización.

#### **RF10 – Cambio de contraseña**

El sistema deberá permitir que los usuarios autenticados cambien su propia contraseña.

#### **RF11 – Usuario MASTER**

El sistema deberá disponer de un usuario con rol MASTER encargado de la administración global del sistema, incluyendo:

- creación de organizaciones
- creación inicial de usuarios
- asignación inicial de contraseñas
- gestión global del sistema.

#### **RF12 – Gestión CRUD de entidades**

El sistema deberá proporcionar operaciones CRUD para:

- usuarios
- organizaciones
- roles

#### **RF13 – Autorización basada en roles (RBAC)**

El sistema deberá implementar un modelo de autorización basado en roles donde los permisos se asignan a roles y los roles a usuarios dentro de una organización.

#### **RF14 – Documentación de la API**

El sistema deberá generar documentación automática de la API mediante Swagger.

### Requisitos no funcionales

#### **RNF1 – Seguridad**

El sistema deberá almacenar las contraseñas de los usuarios utilizando hash seguro.

#### **RNF2 – Autenticación stateless**

El sistema deberá utilizar autenticación basada en JWT, evitando mantener sesiones en el servidor.

#### **RNF3 – Rendimiento**

Las consultas de autorización deberán responder en tiempos bajos, optimizandas mediante índices.

#### **RNF4 – Escalabilidad**

La arquitectura del sistema deberá permitir que el servicio de autorización sea utilizado por múltiples aplicaciones externas.

#### **RNF5 – Modularidad**

El sistema deberá separar claramente la autenticación de un usuario en el sistema de la autorización de las acciones que puede tomar.

#### **RNF6 – Portabilidad**

La aplicación deberá poder desplegarse mediante contenedores Docker.

#### **RNF7 – Persistencia de datos**

El sistema deberá utilizar una base de datos relacional PostgreSQL para almacenar la información del sistema.

#### **RNF8 – Documentación**

La API deberá estar documentada automáticamente mediante Swagger.

#### **RNF9 – Mantenibilidad**

El sistema deberá seguir buenas prácticas de desarrollo para facilitar su mantenimiento y evolución.

#### **RNF10 – Compatibilidad API**

La API deberá seguir el estilo arquitectónico REST.

### Historias de usuario

#### **H1 – Creación de organización**

Como administrador del sistema (MASTER)  
quiero crear una nueva organización  
para permitir que usuarios puedan operar dentro de ella.

#### **H2 – Gestión de usuarios**

Como administrador del sistema  
quiero crear usuarios y asignarles roles  
para controlar qué acciones pueden realizar.

#### **H3 – Consulta de permisos**

Como aplicación externa  
quiero consultar si un usuario tiene permiso para realizar una acción  
para decidir si permitir o denegar la operación.

#### H4 – Cambio de contraseña

Como usuario autenticado  
quiero cambiar mi contraseña  
para mantener la seguridad de mi cuenta.

#### H5 – Autenticación

Como usuario del sistema  
quiero iniciar sesión  
para obtener un token que me permita acceder a la API.

#### H6 – Creación de roles

Como administrador del sistema  
quiero crear roles y asignarles permisos  
para definir clasificar el alcance de los permisos de los usuarios

## Anexo B: Script 01-schema.sql de inicialización del esquema de la base de datos

```
-- USERS: Saves user Login including hashed password
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    pass_hash TEXT NOT NULL
);

-- ORGANIZATIONS: Org. structure, contains a foreign key of itself to allow
for a layer structure
CREATE TABLE organizations (
    ID SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    parent_id INT,

    CONSTRAINT fk_org_parent
        FOREIGN KEY (parent_id)
        REFERENCES organizations(id)
        ON DELETE SET NULL --Orphans child org is parent is deleted,
allows easier restructuring
);

-- ROLES: Description of which roles belong to which orgs
```

```

CREATE TABLE roles (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    description TEXT,
    organization_id INT NOT NULL,

    CONSTRAINT fk_roles_org
        FOREIGN KEY (organization_id)
        REFERENCES organizations(id)
        ON DELETE CASCADE, --if an org is deleted, its roles go with it

    CONSTRAINT unique_role_for_org
        UNIQUE (name, organization_id) --an org cant have two roles with
the same name
);

-- PERMISSIONS: Preset and unmodifiable set of permissions that can be
bundled into roles
CREATE TABLE permissions (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    description TEXT
);

-- ROLES_PERMISSIONS: table describing N-N relation between roles and
permissions
CREATE TABLE roles_permissions (
    role_id INT NOT NULL,
    permission_id INT NOT NULL,

    PRIMARY KEY (role_id, permission_id),
    --If either the permission or role is deleted the relation expires with
it (CASCADE)
    CONSTRAINT fk_rp_role
        FOREIGN KEY (role_id)
        REFERENCES roles(id)
        ON DELETE CASCADE,

    CONSTRAINT fk_rp_permission
        FOREIGN KEY (permission_id)
        REFERENCES permissions(id)
        ON DELETE CASCADE
);

```



```

-- USER_ORG_ROLE: table describing N-N relation between a user and the
roles they have in a given org
CREATE TABLE user_org_role (
    user_id INT NOT NULL,
    organization_id INT NOT NULL,
    role_id INT NOT NULL,

    PRIMARY KEY (user_id, organization_id, role_id),

    --If either the permission, role or org is deleted the relation expires
    with it (CASCADE)

    CONSTRAINT fk_uor_user
        FOREIGN KEY (user_id)
        REFERENCES users(id)
        ON DELETE CASCADE,

    CONSTRAINT fk_uor_org
        FOREIGN KEY (organization_id)
        REFERENCES organizations(id)
        ON DELETE CASCADE,

    CONSTRAINT fk_uor_role
        FOREIGN KEY (role_id)
        REFERENCES roles(id)
        ON DELETE CASCADE
);

```

## Anexo C: Script 02-init-data.sql de insercion de datos iniciales

```

--BASE PERMISSIONS: This is the static list of permissions assignable to
roles
INSERT INTO permissions (name, description) VALUES
('CREATE_USER', 'Create new users'),
('READ_USER', 'Read user data'),
('UPDATE_USER', 'Update user data'),
('DELETE_USER', 'Delete users'),

```

```

('CREATE_ORG', 'Create organizations'),
('READ_ORG', 'Read organization data'),
('UPDATE_ORG', 'Update organization data'),
('DELETE_ORG', 'Delete organizations'),

('CREATE_ROLE', 'Create roles'),
('READ_ROLE', 'Read roles'),
('UPDATE_ROLE', 'Update roles'),
('DELETE_ROLE', 'Delete roles'),

('ASSIGN_ROLE', 'Assign roles to users');

--SYSTEM ORG: this is a global org, only the db admin will be assigned to
it
INSERT INTO organizations (name, description)
VALUES ('SYSTEM', 'Internal system organization');

--MASTER ROLE: has control over the entire database
INSERT INTO roles (name, description, organization_id)
VALUES ('MASTER', 'System administrator with full access', 1);

--assign all premissions to master role
INSERT INTO roles_permissions (role_id, permission_id)
SELECT 1, id FROM permissions;

--create master user
INSERT INTO users (username, email, pass_hash)
VALUES ('master', 'master@system.com', 'TEMP_HASH'); --hash will be
reassigned via backend

--assign role to master user in system org.
INSERT INTO user_org_role (user_id, organization_id, role_id)
VALUES (1, 1, 1);

```